VoltSentry — Presenter Guide

How and when to explain what. A 4-minute run-of-show, a slide-by-slide script, the live-demo runbook, recovery lines, and Q&A prep.

> **The one sentence:** *"VoltSentry is a transparent software proxy that sits on the wire between an EV charger and its management backend, inspects every OCPP message in real time, and quarantines a compromised charger before it can commit billing fraud or destabilise the grid — including the slow data-drift attack that no rule engine can catch."*

Roles & setup

ROLE	OWNS
Narrator	Talks the whole time. Never touches the keyboard. Owns the story and the Q&A.
Driver	Advances slides, switches to the dashboard, points the cursor at what the narrator is describing.
Operator	Presses the attack keys on cue (`1 2 3 4`, `r` between each). Watches the terminals, calls out if a process dies.

Pre-flight checklist (do before you walk up)

- Start order, one terminal each: `python -m simulator.csms` → `python -m proxy.main` → `python -m simulator.twin` → `cd dashboard && npm run dev` → `python -m cli.attack`. The proxy dials upstream at handshake, so CSMS must be up first.
- Dashboard on screen, logged in (Demo Quick-Login), fleet charging, 8 stations green, transformer gauge steady.
- Click **Start Monitoring (enable audio)** once — browser autoplay policy blocks the alert sound until a click.
- Browser at 100% zoom, dark mode, full screen, OS notifications off.
- **Backup video** of one clean 4-attack run open in a background tab. If anything wedges mid-demo, cut to it and keep narrating.
- Terminals tiled where judges can see the `[proxy]` / `[twin]` prefixes — they are part of the proof.
- Run the full sequence once, cold, five minutes before you present.

The 4-minute timeline

TIME	ON SCREEN	BEAT
0:00	Deck slide 1–4	Problem → threat model → the approach. Fast.
0:50	Deck slide 5–9	Architecture, real units (the 230 V vs 415 V answer), the multi-site network, then the two detection tiers. Land "the residual feature."
1:45	Dashboard	Switch to the live NOC. Fleet charging.
2:00	Dashboard	Press `1` — meter_spoof. CP-03 quarantined.
2:30	Dashboard	`r` then `2` — oscillation. Transformer load swings 82 % ↔ 0 % every second.
2:55	Dashboard	`r` then `3` — session shadow. Handshake refused.
3:15	Dashboard	`r` then `4` — subtle drift. **No rule fires.** Risk Index climbs past 0.65.
3:35	Dashboard	Attack 5 — card clone across sites; show the **Multi-Site Network** tab. (Only if the two-site network is running.)
3:50	Dashboard	Reset **both** sites, then click **Export incident logs** — JSON downloads.
3:55	Deck slide 11 / 14	"What's real" honesty beat, then the close.

Slide-by-slide script

Talking points are prompts, not a script to read aloud. Say them in your own words.

1 Title

One point: what this is, in one breath.

- "EV charging networks run on a protocol called OCPP — JSON messages over a WebSocket between the charger and a central server."
- "VoltSentry is an inline proxy that reads every one of those messages and stops a compromised charger before the damage reaches the backend or the grid."

2 The problem

One point: this is exposed infrastructure, and today's defences are too slow or too expensive.

- Chargers are roadside hardware on weakly-authenticated connections — soft edge nodes on the energy grid.
- Cloud platforms analyse this data in batches, minutes to hours later — after an attack has already landed.
- You can't retrofit a firewall chip to millions of installed chargers. So the fix has to be pure software, and it has to be inline.

3 Threat model

One point: five concrete attacks, and the drift is special.

- Meter spoofing — bill near zero while real power flows. Grid oscillation — synchronised start/stop to trip a transformer. Session shadowing — hijack a live charger's session with a duplicate handshake.
- "Subtle drift is a slow, compounding lie that never breaks a single threshold. Hold that one in mind; it's why the second detection tier exists."
- Card cloning — a copied RFID card charging two cars at once, often at two different sites. No single site can see it.

4 The approach

One point: transparent, inline, drop-in — the CSMS never knows it's there.

- A reverse proxy on the wire. Clean traffic is relayed straight through, both directions, untouched.
- Every frame is parsed, the session state machine is enforced, and the detection engine scores it before it's forwarded.
- On a hard violation: drop the frame, tell the charger to go unavailable, close the socket. Sub-millisecond design budget, zero hardware change.

5 Architecture

One point: one process on the wire, feeding one live dashboard.

> DO – Trace the diagram left to right with the cursor: fleet → proxy → CSMS, and the side branch proxy → dashboard + forensic log.

- Station identity comes from the URL path, parsed at handshake — the proxy opens a matching upstream socket per charger.
- OCPP replies don't carry the action name, so the proxy keeps its own call-correlation map. That's what makes the state machine possible.
- The dashboard feed is a separate WebSocket on its own port — the browser talks to it natively.

6 Real units

One point: every number is in its real unit, and here is the supply chain.

- "A house takes one phase at about 230 volts. A fast charger needs all three phases: 415 volts between lines, 240 from each line to neutral."
- "The 1250 is the transformer's rating in kVA, not volts. Eight 120-kilowatt chargers need about 1022 kVA, so 1250 is the next standard size. Our hackathon build said 500 kVA, which they would have overloaded twice over; we fixed it."
- "The dashboard shows what the chargers measure: bus voltage both ways, line current, frequency. And a new rule, R7, checks that reported power equals volts times amps."

7 Multi-site network

One point: sites protect themselves, and together they see what no single site can.

- "Each charging site runs its own VoltSentry node. They connect to a hub that shares threats and blocklists; every message is signed with that site's own key."
- "If a charger is quarantined in Vellore, Chennai refuses it. If one card is charging in both cities at once, it's cloned, and it's blocked."
- "If the hub goes down, each site keeps protecting itself."

8 Tier-1 — deterministic rules

One point: nine rules, zero latency, they catch everything loud.

- "These are physics and protocol invariants — power can't exceed the charger's rating, a session can't send meter values before it's authorised, the same charger ID can't hold two live sessions."
- Point at `R6_METER_FRAUD`: "This one watches the reported energy total pull away from the reported power. Honest sessions keep those in lock-step; a skimming charger can't."
- Point at `R7_ELECTRICAL`: "The meter reports power, voltage and current together. Power has to equal volts times amps. A charger that edits only the power figure contradicts itself."
- "Four of these quarantine the station outright — R2, R5, R6, R7. R8 and R9 block a cloned or blocklisted RFID card instead, because the card is compromised, not the charger."

9 Tier-2 — the drift problem

One point: the residual feature is the whole trick.

- "A slow drift stays inside every threshold — so it's not an outlier, and a rule engine is blind to it by construction."
- "We give the model one engineered number: the reported energy total, minus the integral of reported power over time. Honest sessions hold that at zero. Any lie makes it climb — and it only ever climbs."
- "Two models score it: an Isolation Forest on the session vector, and a CUSUM change-detector on the residual for a second opinion and a plain-English explanation."

10 Demo intro

One point: set expectations, then switch to the dashboard.

- "Five keystrokes, five attacks. Watch the station grid and the Risk Index bar on each card."
- "The last one is the one to watch — no rule will fire, and the system still catches it."

`DO` – Driver switches to the live dashboard now. Leave the deck; you'll come back for slide 11.

Live-demo runbook

Operator presses keys in `cli/attack`. Always press `r` and wait for the fleet to go green again between attacks.

Attack 1 — `1` — meter_spoof

EXPECTED	CP-03 reports ~5 kW while its own meter still reads ~718 V × 167 A ≈ 120 kW. On the **first** spoofed frame → `R7_ELECTRICAL` → card goes red, frame dropped, **Quarantined** at 0 kW, audio alert. Risk Index on CP-03 also spikes (Tier-2 agrees).

> - "CP-03 is now claiming five kilowatts. But in the same message its meter says 718 volts at 167 amps. That's 120 kilowatts. The charger contradicted itself."
> - "R7 caught it on one frame. The proxy dropped the frame, told the charger to go offline, and cut the socket. That station is contained."

> IF IT MISBEHAVES — R7 needs the charger to report voltage and current; the twin always does. If the card is not red within a second, say "the energy residual is accumulating" — `R6_METER_FRAUD` trips within ~8 s as the backstop — then move on to `r` and attack 2.

Attack 2 — `r`, then `2` — fleet_oscillate

EXPECTED	Every bay stops and starts together, once a second. The site transformer swings between ~82 % load (about 1,020 kVA) and 0 % every second, and the bay cards flip between **Charging** 120 kW and **Finishing** 0 kW. After ≥ 6 transitions in 10 s → `R3_OSCILLATION` fires fleet-wide (one badge, not eight), about 1–2 s in.

> - "This is a coordinated attack — synchronised start/stop across the cluster to swing local load and trip a distribution transformer."
> - "R3 counts transitions in a sliding window. It's fleet-scoped on purpose — one finding, not noise."

> IF IT MISBEHAVES — If the gauge swing is subtle, point at the transition count / the R3 badge instead of the gauge.

Attack 3 — `r`, then `3` — session_shadow

EXPECTED	A second WebSocket opens to a charger ID that already has a live session. The proxy refuses it at the handshake → `R4_SESSION_UNIQUE`. The legitimate session is untouched.

- "Someone just opened a parallel connection claiming to be CP-01, which is already online. That's a session hijack attempt."
- "The proxy keeps a registry of live connections and rejects the duplicate before a single frame is exchanged. The real charger never notices."

Attack 4 — `r`, then `4` — subtle_drift (the pitch)

EXPECTED	One station under-reports power by a compounding ~2% per tick, and scales its current to match, so every reading agrees with every other. **No Tier-1 rule fires**. The Risk Index crosses 0.65 about **10 s** in (measured) → amber **ML ANOMALY** badge. About 35 s later (~44 s in) the residual is big enough for `R6_METER_FRAUD` to quarantine it.

- "Every number this station is sending is inside spec. No rule can touch it."
- "But the energy residual is climbing, and the model sees it. Watch the Risk Index — there it goes past the line. That's the attack a threshold engine can't catch, caught."

IF IT MISBEHAVES — Keep talking for the ~10 s; the amber badge is the moment. If it lands while you are mid-sentence, point back at it: *"there — the model flagged it, no rule involved."* The R6 quarantine that follows confirms it.

Attack 5 — card clone (needs the two-site network: `python scripts/run_network.py`)

RUN	`python -m cli.attack -a 5 -s CHN-02 -t TAG-CP-05 --control-url ws://localhost:9101/control`
EXPECTED	A Chennai bay presents a card that is still charging a car in Vellore. Chennai's node answers the Authorize itself with **Blocked** → `R8_TAG_CLONE`; the CSMS never sees it. Vellore raises `R9_THREAT_INTEL` on CP-05. The NOC's **Multi-Site Network** tab shows the shared indicator (a hash, not the card number). CHN-02 keeps retrying the copy every 10 s; every try is refused, but it is one incident, so it is logged once a minute, not every retry. The real car on CP-05 keeps charging.

- "One card can only charge one car at a time. Neither site alone can see this card is in two places — the network can."

Close the demo — reset both sites

DO — The clone runs on Chennai, so reset both sites: press `r` in the CLI (Vellore), then run `python -m cli.attack -a r --control-url ws://localhost:9101/control` (Chennai). Wait a few seconds until the **Multi-Site Network** tab shows every bay charging at both sites. Then click **Export incident logs** — a JSON file downloads. "Every detection is on the wire and in the forensic log."

EXPECTED	The cloned card stays on the network blocklist for an hour; a reset does not unblock it. So when CP-05 starts again, its old card `TAG-CP-05` is refused once, the operator issues the driver a replacement (`TAG-CP-05-R1`), and CP-05 charges on the new card a few seconds later. That refusal adds at most one `R9_THREAT_INTEL` line; after it the security log stays quiet.

> - If asked why CP-05's card changed: "Once a card is copied, the copy is as good as the original, so the card number is blocked everywhere and the customer gets a new card. That's what an operator does with a cloned bank card too."

Known risks & recovery

RISK	RECOVERY
A socket drops mid-demo	The dashboard auto-reconnects every 2 s — keep talking, it comes back. If it doesn't in ~5 s, cut to the backup video.
Audio alert silent	You forgot the **Start Monitoring** click in pre-flight. Don't stop to fix it — the visual is enough.
A process died	Operator says so quietly; Narrator keeps on the deck; restart in order CSMS → proxy → twin, or cut to the backup video.
The Multi-Site tab says "hub not reachable"	The network was started single-site. Skip attack 5, or start it with `python scripts/run_network.py`.
"Reset" leaves a station stuck	Press `r` again; it re-runs Boot → Authorize → StartTransaction and resynchronises both sides.
CHN-02 not charging after the clone demo	Only Vellore was reset; CHN-02 is still presenting the copied card. Reset Chennai too: `python -m cli.attack -a r --control-url ws://localhost:9101/control`.
Running attack 5 a second time	`TAG-CP-05` is already blocked, so cloning it again shows `R9_THREAT_INTEL` instead of `R8_TAG_CLONE`. Clone a different card (`-t TAG-CP-06`), or restart `python scripts/run_network.py` for a clean network; blocklists are held in memory.

Q&A preparation

"Isn't your simulation too simple? Is any of this real?"

The **protocol and the detection are real** — real OCPP 1.6-J framing, real call correlation, real session state machine, the actual rule and ML logic, the real quarantine mechanics. What's simulated is the **charger physics** and the **CSMS backend** — and the backend is a drop-in: point the proxy at SteVe or CitrineOS, both real open-source management systems, and nothing changes on our side. The attacks are synthesised because there is no public labelled corpus of OCPP attacks — that's true of the whole field.

"Why an Isolation Forest? Why not a deep model / LSTM?"

The reasoning architecture is the **two tiers**, not the estimator. For a five-dimensional conservation-law residual, a small model we can prove correct beats a big one we can't — and an LSTM in the forwarding path costs 100–1000× the inference latency, which breaks the "inline, sub-millisecond" claim. If we ever add a temporal model it runs out-of-band on a mirrored stream, off the critical path.

"Does this actually scale?"

Tier-1 evaluation is ~16 microseconds per full MeterValues frame (eleven readings, about 1 KB), measured over 200k frames. One instance sustains thousands of frames per second; a 10,000-charger operator at one meter reading every 10 seconds is about 1,000 frames/sec — a handful of instances. Horizontal scale is charger-ID consistent hashing across replicas with the registries in Redis.

"What about TLS and authentication?"

TLS is available on the charger link and on the links between sites (verified with TLS 1.3); the laptop demo runs plaintext by default. Every message between VoltSentry nodes is HMAC-signed with that node's own key and replay-protected. The OCPP security profiles (password and client-certificate auth for chargers) are next. The detection value is orthogonal anyway: an authenticated charger can still be compromised and lie.

"Homes get 230 V. Why does your transformer say 1250, and why not 220 V?"

The 1250 is the transformer's rating in **kVA** — apparent power, not volts. A house uses one phase at about 230 V. A 120 kW charger needs three-phase power: 415 V between lines, which is 240 V from each line to neutral, the same order as a home. Eight 120 kW chargers draw about 1022 kVA from an 11 kV feeder, so the site needs the next standard size, 1250 kVA. The dashboard shows the measured bus voltage both ways, the line current (about 1400 A per phase at full load), and the frequency.

"How do several VoltSentry servers work together?"

Each charging site runs its own VoltSentry node beside its chargers, and every node dials out to a federation hub. The hub relays threats and blocklists between sites and keeps a table of which RFID cards are charging where, as salted hashes. A charger quarantined in Vellore is refused if it shows up in Chennai, and a card charging in two places at once is blocked. If the hub goes down, each site keeps protecting itself; the queued intel is delivered when it returns. We tested forged keys, impersonation and replayed messages: all refused.

"What firewall are you using?"

Show: the Defences tab. VoltSentry *is* the firewall: an application-level firewall that reads OCPP, built by us in Python, no third-party product. A network firewall decides by address and port, so it cannot see a charger reporting 5 kW while drawing 120 kW; VoltSentry reads inside every message and holds back the forged ones. The diagram at the top shows where it sits — chargers → the site's own network firewall → VoltSentry → CSMS — with live counts of messages inspected, forwarded and held back. A real site uses both firewalls; in the laptop demo there is no network firewall.

"Apart from quarantine, what other safeguards do you have?"

Show: the Defences tab, scroll down. One card per safeguard, with its live count: duplicate connections refused (R4), blocklisted chargers refused (R9), oversized messages refused (64 KB), malformed connections refused, forged messages held back from the CSMS, cloned cards answered *Blocked*, signed site links (HMAC-SHA256, per-site keys), replay protection, card numbers shared only as salted hashes, a blocklist that expires, monitor mode and the forensic log. TLS is available but off on the laptop (the card says "lab set-up"). Charger passwords/certificates and real console login are shown as **planned** — say so before you are asked.

"How is this different from a web application firewall?"

A WAF doesn't speak OCPP. VoltSentry enforces the **session state machine** (you can't send meter values before you're authorised), **physical invariants** (power can't exceed the charger's rating), and **energy conservation** across a session. Those are domain rules a generic firewall has no concept of.

"How does R6 catch meter fraud without false-positiving honest sessions?"

It requires *both* a meaningful accumulated residual *and* a residual that is still climbing fast right now. A brief measurement wobble produces neither. Sustained under-reporting produces both, and the gap only widens.

"What's your false-positive rate?"

On the simulated fleet: none of 28 full honest sessions (from arrival to 100 %, up to 2 hours each) crossed 0.65, and a 25-minute live run with every car going through the 80 % hand-off raised no alarm. That took a fix: the first model was trained only on full-power charging and flagged every honest car above 80 %; it is now trained on the whole charging curve and its

scale is calibrated on separate honest sessions. Be honest about the rest: on **real** data the rate is not measured yet. Tier-1 rules are physical and protocol invariants, so an honest charger should not trip them; Tier-2 needs re-fitting on a real dataset (ACN-Data or ElaadNL), and monitor mode exists so a site can measure it on its own meters before blocking anything.

"Who buys this, and how is it deployed?"

Charge-point operators (CPOs) and CSMS vendors. It deploys as an ingress sidecar in front of the existing CSMS — no charger firmware change, no backend change. That's the entire commercial pitch: security you can add without touching the fleet.

"What's genuinely novel here?"

Inline, transport-layer, OCPP-aware detection with active quarantine — and specifically the energy-conservation residual as a feature that makes slow billing fraud detectable from the wire alone, without ground-truth power measurement.

VoltSentry · CodeToCreate, VIT Vellore · Generated for the team. Numbers in this guide trace to the codebase and its test suite; update them if the code changes before you present.